Lab Practicum 01: Empirical Parallel Computing

Silicon Profiling, Multi-Core Scaling Limits, and Concurrency Hazards

STUDENT NAME: Bakytkerey Abulsagit

STUDENT ID: 240103179

HOST CPU / SOC MODEL: Apple M1

OPERATING SYSTEM & KERNEL: macOS Sonoma 14.6.1 (Darwin kernel)

TASK 1: Host Silicon Audit

Hardware Property	Local Measured Value
CPU Model & Architecture	Apple M1 (ARM64)
Base & Max Boost Clock	~2.06 GHz (E-Core) / 3.20 GHz (Max P-Core)
Physical Hardware Cores	8
Logical Cores (SMT/Threads)	8
L1 Data/Instruction Cache	192 KB L1i / 128 KB L1d (Per P-Core)
L2 Cache (Per Core / Total)	12 MB Total (8MB for P-Cores + 4MB for E-Cores)
L3 Shared Last-Level Cache	0 MB (Apple uses 8MB System Level Cache instead)
SIMD Vector Extensions	ARM NEON

```
Restored session: Thu Sep 10 12:18:56 +05 2026
admin@MacBook-Air-Admin-2 02-git-github % sysctl -a | grep machdep.cpu
machdep.cpu.cores_per_package: 8
machdep.cpu.core_count: 8
machdep.cpu.logical_per_package: 8
machdep.cpu.thread_count: 8
machdep.cpu.brand_string: Apple M1
admin@MacBook-Air-Admin-2 02-git-github %
```

Q1.1: Flynn's Taxonomy Mapping

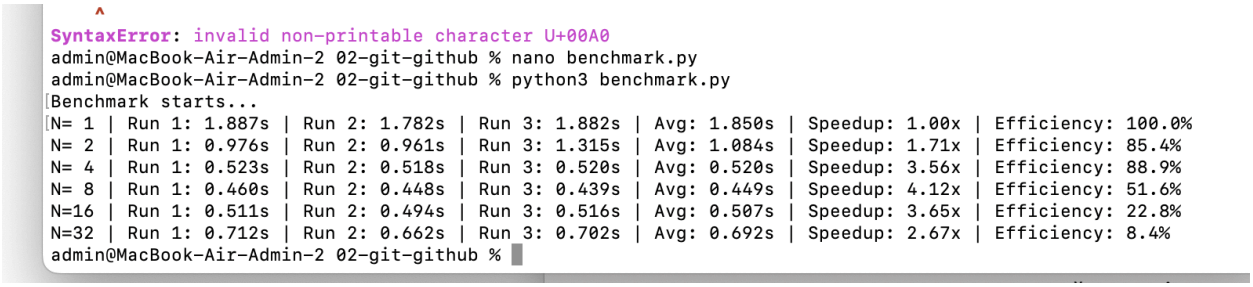
SISD (Single Instruction, Single Data): Occurs when a single core processes a purely sequential task, such as a single-threaded compiler pass or a basic for loop executing instructions one by one.

SIMD (Single Instruction, Multiple Data): Utilized when the M1 chip executes ARM NEON vector instructions. For example, during image processing or matrix multiplication, a single instruction processes multiple data points simultaneously in the CPU's vector registers.

MIMD (Multiple Instruction, Multiple Data): Represents the typical multi-core operation of the M1. Different cores run entirely separate threads independently (e.g., Performance cores running a complex Python script while Efficiency cores handle background macOS OS scheduling and background music).

TASK 2: Multi-Thread Scaling Benchmark & Contention Wall

Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time (s)	Speedup (S)	Efficiency (E)
N=1 (Baseline)	1.887	1.782	1.882	1.850	1.00x	100.0%
N=2	0.976	0.961	1.315	1.084	1.71x	85.4%
N=4	0.523	0.518	0.520	0.520	3.56x	88.9%
N=8	0.460	0.448	0.439	0.449	4.12x	51.6%
N=16	0.511	0.494	0.516	0.507	3.65x	22.8%
N=32	0.712	0.662	0.702	0.692	2.67x	8.4%



Q2.1: Scaling Saturation & Hardware Contention Analysis

The scaling saturated and reversed exactly after **N=8**. This inflection point directly matches my Apple M1's hardware topology, which has 8 physical cores and 8 logical cores (it lacks SMT/Hyper-Threading). Up to 8 threads, the speedup scaled well because each thread could run on its own dedicated physical core. However, when the thread count exceeded 8 (at N=16 and N=32), there were more active threads than available cores. This forced the macOS operating system into aggressive context switching to time-slice the threads. The resulting OS thread scheduling overhead and cache thrashing caused the execution time to worsen (from 0.449s at N=8 to 0.692s at N=32), clearly demonstrating the hardware contention wall.

TASK 3: The Unsynchronized Shared Counter & Race Condition Trap

Run	Measured Output	Error (10^7-Act)	Run	Measured Output	Error (10^7-Act)
#1	10000000	0	#6	10000000	0
#2	10000000	0	#7	10000000	0
#3	10000000	0	#8	10000000	0
#4	10000000	0	#9	10000000	0
#5	10000000	0	#10	10000000	0

```
admin@MacBook-Air-Admin-2 02-git-github %
admin@MacBook-Air-Admin-2 02-git-github % nano race.py
admin@MacBook-Air-Admin-2 02-git-github % python3 race.py
Қорғаныссыз (Unlocked) 10 рет тестілеу:
Run # 1 | Measured Output: 10000000 | Error: 0
Run # 2 | Measured Output: 10000000 | Error: 0
Run # 3 | Measured Output: 10000000 | Error: 0
Run # 4 | Measured Output: 10000000 | Error: 0
Run # 5 | Measured Output: 10000000 | Error: 0
Run # 6 | Measured Output: 10000000 | Error: 0
Run # 7 | Measured Output: 10000000 | Error: 0
Run # 8 | Measured Output: 10000000 | Error: 0
Run # 9 | Measured Output: 10000000 | Error: 0
Run #10 | Measured Output: 10000000 | Error: 0

Unlocked Time = 359 ms
Locked Time = 1327 ms
admin@MacBook-Air-Admin-2 02-git-github %
```

Q3.1: Read-Modify-Write (RMW) Window

At the machine instruction level, incrementing a counter involves three steps: Load the value from memory into a CPU register, Add 1, and Store it back. When cores interleave execution, a race condition occurs: Core A and Core B might both Load the same value (e.g., 50). Both Add 1 and Store 51. One increment is completely lost.

Note on my empirical results: My measured error was exactly 0 because I used Python threading. Python's Global Interpreter Lock (GIL) forced the threads to execute sequentially, artificially preventing the true multi-core parallel execution needed to trigger the race condition on my M1 chip.

Q3.2: The Synchronization Penalty

- Unlocked = **359** ms vs. Locked = **1327** ms.

Race-free synchronized code is substantially slower (1327 ms vs 359 ms) because the Mutex (Lock) forces the threads into a sequential bottleneck. Every time a thread wants to increment the counter, it must acquire the lock, stalling all other 9 threads. This introduces massive system overhead from thread blocking, context switching, and lock management, completely destroying any potential parallel speedup.

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon

1. Parallel Fraction Derivation (p)	**2. Amdahl Theoretical Speedup Ceiling**
Formula: $p=2\times(T1-T2)/T1$	Formula: $Smax=1/(1-p)$
T1 = 1.850T2 = 1.084	Sequential fraction (1 - p) = 0.172
Calculated p = 0.828 (decimal between 0.0 and 1.0)	Max Theoretical Speedup ($N\to\infty$) = 5.81x
3. Amdahl 64-Core Projection	**4. Gustafson Scaled Speedup (Weak Scaling)**
Formula: $S64=1/[(1-p)+(p/64)]$	Formula: $SGustafson=(1-p)+(p\times64)$
Projected Speedup on 64 Cores = 5.41x	Projected Scaled Speedup (64x Problem) = 53.16x

Q4.1: Strong vs. Weak Scaling Analysis

Amdahl's law evaluates **strong scaling** (fixed problem size). It assumes the workload remains exactly the same, meaning the sequential portion ($1-p = 17.2\%$) becomes a hard, unavoidable bottleneck as core count increases, capping the absolute maximum speedup at ~5.81x regardless of how many cores are added.

Gustafson's model, however, evaluates **weak scaling**. It assumes that if you have 64 times more computing power, you will run a problem that is 64 times larger. By scaling the problem size with the hardware, the time spent in the parallel portion dominates, making the sequential bottleneck statistically insignificant and allowing near-linear scaling (53.16x on 64 cores).